Better return types in `std::inplace_vector` and `std::exception_ptr_cast`

Document #:	P3981R2 [Latest] [Status]
Date:	2026-03-26
Project:	Programming Language C++
Audience:	LWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com> Jonathan Wakely <cxx@kayari.org> Tomasz Kamiński <tomaszkam@gmail.com>

Contents

§ 1 Revision History

Since [P3981R1]: Wording update.

Since [P3981R0]: discussion at a LEWG telecon lead to deciding to remove `try_append_range` instead of modifying it. See [P4022R0] (Remove `try_append_range` from `inplace_vector` for now).

§ 2 Introduction

This paper seeks to address the following NB comments:

- PL-006
- US 68-122

— US 150-228

— GB 08-225

The new C++26 container `std::inplace_vector<T, N>` contains four functions or function templates which conditionally try to perform some operation, which might fail due to exceeding capacity. Those signatures are currently:

```
template<class... Args>
  constexpr pointer try_emplace_back(Args&&... args);
constexpr pointer try_push_back(const T& x);
constexpr pointer try_push_back(T&& x);

template<container-compatible-range<T> R>
  constexpr ranges::borrowed_iterator_t<R> try_append_range(R&& rg);
```

We argue in this paper that there is a better choice for the return type for each of these algorithms: `optional<reference>` for the first three ~~`and ranges::borrowed_subrange_t<R>`~~ ~~for the fourth~~.

We are aware that [P3739R4] (Standard Library Hardening - using std::optional) exists, but we feel that this is an important change to make, and that paper's motivation is weak, has a misleading title (it has nothing to do with standard library hardening), and argues for a return type of `optional<T&> const`, which is not a good idea.

§ 3 `T*` makes for a poor `optional<T&>`

The functions `emplace_back` and `push_back` return a `reference` to the new element that was added to the container. The functions `try_emplace_back` and `try_push_back` seek to do the same thing, except that these functions signal failure via the return path: they can only *conditionally* return a reference to the element that was added, so they need to return something else on failure.

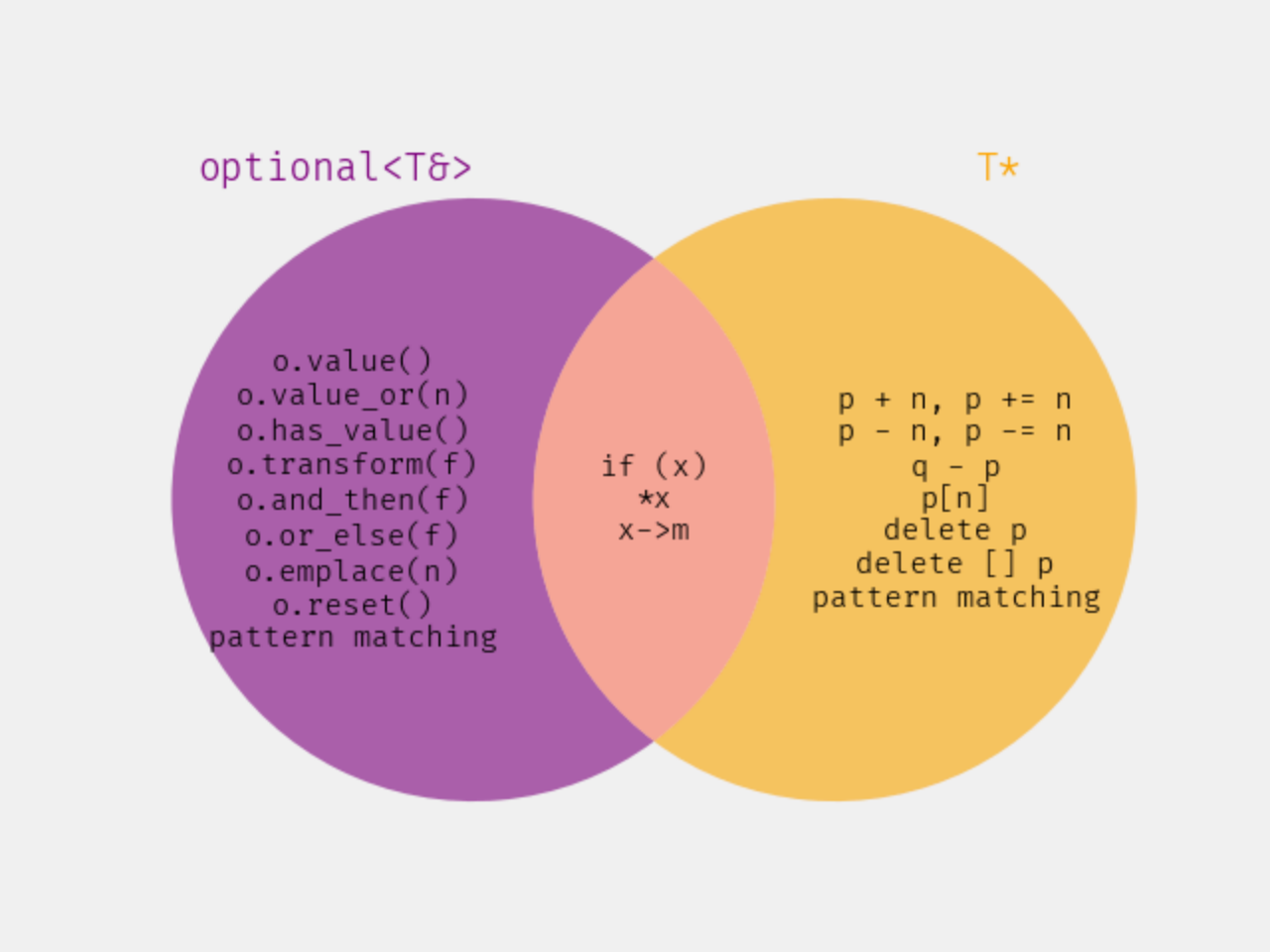
When [P0843R14] (inplace_vector) was adopted, the only sensible choice for the return type was `T*`: either a pointer that points to the new element or a null pointer. However, now that [P2988R12] (std::optional<T&>) was adopted for C++26, there is another choice: `optional<T&>`. These types are, superficially, quite similar to each other. And, indeed, whenever the idea of an optional reference comes up, inevitably somebody will bring up the point that we don't need `optional<T&>` because we already have `T*`. Barry wrote a whole blog post several years about about how `T*` makes for a poor `optional<T&>` responding to this claim.

There are several significant benefits to `optional<T&>` when it comes to the return type here, which we will enumerate.

First, as you can already see in the opening paragraph of this section, in some ways `optional<T&>` is already inherently the correct choice. We have `push_back` returning a `T&`. So `try_push_back`, which instead of having a precondition simply tries and might fail, should return an `optional<T&>`. That is the general shape of fallible functions.

Second, consider the semantics. `T*` has many possible semantics: it is either an owning or non-owning pointer, that could point to a single object, to an array of objects, or past-the-end of an object or array. In our case, we are returning either a reference to a single object or nothing — which is exactly the singular semantic of `optional<T&>`.

Third, consider the potential API of the return type:



There are some operations that `T*` and `optional<T&>` share in the middle, those have the same semantics and meaning either way. All of the operations in the purple circle are highly relevant and useful to this problem. We want to have an optional reference, so it is useful to have the chaining operations that give us a different kind of optional, or to provide a default value, or to emplace or reset, or even to have a throwing accessor. But all the operations in the orange circle are highly *irrelevant* to this problem and would be completely wrong to use. They are bugs waiting to happen. We we don't have an owning pointer, so neither `delete` nor `delete []` are valid. We happen to have an array, but the indexing operations are pretty questionable (are people going to use negative indices here?). Nevertheless, these operations will actually compile.

You'll note that "pattern matching" appears in both circles, differently — this is because [P2688R5] (Pattern Matching: `match` Expression) supports matching both `optional<U>` and `T*`, but they are matched *differently*:

— an `optional<U>` matches against `U` or `nullopt`, because that's precisely what it represents.

— a `T*` doesn't match against a `T`, rather it matches polymorphically.

We still won't have pattern matching in C++26, but if we ever do, we'll want to be able to match on whether our optional reference actually contains a reference, or not. We do not need to match whether we're holding a derived type or not.

Fourth, with standard library hardening, we know that `*x` and `x->m` will be checked if `x` is an `optional<T&>`. But there is no such guaranteed checking for raw pointers. Hopefully, you segfault?

These are very significant benefits to returning `optional<T&>`. There are simply no benefits to returning `T*`.

§ 3.1 The same holds for `std::exception_ptr_cast<E>`

Similarly, `std::exception_ptr_cast<E>(p)` was introduced by [P2927R3] (Observing exceptions stored in exception_ptr), and the exact same arguments hold. That function wants to either return an object or nothing. `optional<E const&>` is simply a better return type than `E const*` here. With the additional argument that here even the indexing operations are undefined behavior, whereas for the `std::inplace_vector` case they are simply questionable.

§ 3.2 Other related functions

There are two other, closely related functions in the standard library that each conditionally return a reference to an object: `std::any_cast` (the form that returns a pointer) and `std::get_if` (for `std::variant`). For both of these functions, `std::optional<T&>` didn't exist yet, so the only option was `T*` (and we are not suggesting changing them now), but this behavior has proved quite clunky in practice. Now that we have a better option, we should use it.

§ 4 Iterator or Subrange?

A previous revision of this paper made the argument that `v.try_append_range(r)` should return a `subrange` instead of an `iterator` (as pointed out by PL-006). But we've decided it is best to remove the function entirely for now, see [P4022R0] (Remove `try_append_range` from `inplace_vector` for now). This paper no longer proposes any changes.

§ 5 Why not do this?

[P3830R0] (NB-Commenting is Not a Vehicle for Redesigning inplace_vector) argues that we simply should not make this change, mostly on the basis that it is new. Which, yes, the specific specialization `std::optional<T&>` is new, and it took unnecessarily long to adopt it after `std::optional<T>` was adopted (despite its existence in Boost for decades, and proliferation across many other `optional` implementations). But the notion of an optional reference in general is not new, and we have a lot of experience with it outside of the standard library — and even outside of C++. The Rust standard library returns optional references from many APIs quite liberally.

The paper additionally points out several issues with `std::optional<T&>` specifically, which are:

— [LWG4299] and [LWG4300] are basically typos and were already both fixed.

— [LWG4308] is more interesting, but not really relevant to the question of whether to use it in `std::inplace_vector`, and was already fixed.

— That `optional<T&>` wasn't explicitly specified to be trivially copyable was also basically a typo, and was also already fixed by [P3836R2] (Make optional<T&> trivially copyable (NB comment US 134-215)).

None of which sound to us like reasons to not make these changes.

§ 6 Proposal

We propose to change the return types of ~~four~~ three algorithms in `std::inplace_vector<T, N>`: `try_emplace_back` and both overloads of `try_push_back` to return `optional<T&>` instead of `T*` And to likewise change the return type of `std::exception_ptr_cast` from `E const*` to `optional<E const&>`.

§ 6.1 Wording

Change 17.9.2 [exception.syn]:

```
// all freestanding
namespace std {
  // ...
- template<class E> constexpr const E* exception_ptr_cast(const excep-
        tion_ptr& p) noexcept;
+ template<class E> constexpr optional<const E&> exception_ptr_cast(con-
        st exception_ptr& p) noexcept;
   template<class E> void exception_ptr_cast(const exception_ptr&&) =
        delete;
  // ...
}
```

Change 17.9.7 [propagation]/14-15:

```
- template<class E> constexpr const E* exception_ptr_cast(const excep-
        tion_ptr& p) noexcept;
+ template<class E> constexpr optional<const E&> exception_ptr_cast(con-
        st exception_ptr& p) noexcept;
```

14 *Mandates*: […]

15 *Returns*: ~~A pointer to the~~ An `optional` containing a reference to the exception object referred to by `p`, if `p` is not null and a handler of type `const E&` would be a match ([except.handle]) for that exception object. Otherwise, ~~`nullptr`~~ `nullopt`.

Change the synopsis in 23.3.16.1 [inplace.vector.overview]:

```
namespace std {
  template<class T, size_t N>
  class inplace_vector {
  public:

    // ...

    // [inplace.vector.modifiers], modifiers
    template<class... Args>
                constexpr   reference   emplace_back(Args&&...   args);
        // freestanding-deleted
                  constexpr    reference    push_back(const    T&    x);
        // freestanding-deleted
                     constexpr     reference     push_back(T&&     x);
        // freestanding-deleted
    template<container-compatible-range<T> R>
                          constexpr    void    append_range(R&&    rg);
        // freestanding-deleted
    constexpr void pop_back();

    template<class... Args>
-     constexpr pointer try_emplace_back(Args&&... args);
+     constexpr optional<reference> try_emplace_back(Args&&... args);
-   constexpr pointer try_push_back(const T& x);
-   constexpr pointer try_push_back(T&& x);
+   constexpr optional<reference> try_push_back(const T& x);
+   constexpr optional<reference> try_push_back(T&& x);
    template<container-compatible-range<T> R>
      constexpr ranges::borrowed_iterator_t<R> try_append_range(R&& rg);

    // ...
  };
}
```

Change 23.3.16.5 [inplace.vector.modifiers]/8-11:

```
  template<class... Args>
-   constexpr pointer try_emplace_back(Args&&... args);
+   constexpr optional<reference> try_emplace_back(Args&&... args);
- constexpr pointer try_push_back(const T& x);
- constexpr pointer try_push_back(T&& x);
+ constexpr optional<reference> try_push_back(const T& x);
+ constexpr optional<reference> try_push_back(T&& x);
```

8 Let vals denote a pack: […]

9 *Preconditions*: […]

10 *Effects*: […]

11 *Returns*: ~~nullptr~~ nullopt if size() == capacity() is true, otherwise ~~addressof(back())~~ optional<reference>(in_place, back()).

§ 6.2 Feature-Test Macros

Bump the two relevant macros in 17.3.2 [version.syn]:

```
- #define __cpp_lib_inplace_vector                    202406L // also in
        <inplace_vector>
+ #define __cpp_lib_inplace_vector                    2026XXL // also in
        <inplace_vector>

- #define __cpp_lib_exception_ptr_cast                 202506L  // also
        in <exception>
+ #define __cpp_lib_exception_ptr_cast                 2026XXL  // also
        in <exception>
```

§ 7 References

[LWG4299] Giuseppe D'Angelo. Missing Mandates: part in optional<T&>::transform.
https://wg21.link/lwg4299

[LWG4300] Giuseppe D'Angelo. Missing Returns: element in optional<T&>::emplace.
https://wg21.link/lwg4300

[LWG4308] Jiang An. std::optional<T&>::iterator can't be a contiguous iterator for some T.
https://wg21.link/lwg4308

[P0843R14] Gonzalo Brito Gadeschi, Timur Doumler, Nevin Liber, David Sankel. 2024-06-26. inplace_vector.
https://wg21.link/p0843r14

[P2688R5] Michael Park. 2025-01-13. Pattern Matching: `match` Expression.
https://wg21.link/p2688r5

[P2927R3] Gor Nishanov, Arthur O'Dwyer. 2025-05-19. Observing exceptions stored in exception_ptr.
https://wg21.link/p2927r3

[P2988R12] Steve Downey, Peter Sommerlad. 2025-04-04. std::optional<T&>.
https://wg21.link/p2988r12

[P3739R4] Jarrad J Waterloo. 2025-11-07. Standard Library Hardening - using std::optional.
https://wg21.link/p3739r4

[P3830R0] Nevin Liber. 2025-09-04. NB-Commenting is Not a Vehicle for Redesigning inplace_vector.
https://wg21.link/p3830r0

[P3836R2] Jan Schultke, Nevin Liber, Steve Downey. 2025-11-06. Make optional<T&> trivially copyable (NB comment US 134-215).
https://wg21.link/p3836r2

[P3981R0] Barry Revzin, Jonathan Wakely, and Tomasz Kamiński. 2026-01-27. Better return types in `std::inplace_vector` and `std::exception_ptr_cast`.
https://wg21.link/p3980r0

[P3981R1] Barry Revzin, Jonathan Wakely, Tomasz Kamiński. 2026-02-23. Better return types in std::inplace_vector and std::exception_ptr_cast.

https://wg21.link/p3981r1

[P4022R0] Barry Revzin, Jonathan Wakely, and Tomasz Kamiński. 2026-02-22. Remove `try_append_range` from `inplace_vector` for now.

https://wg21.link/p4022r0